

UNIVERSIDADE FEEVALE
CURSO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA: PROCESSAMENTO PARALELO

TRABALHO PRÁTICO II
SOMA DE MATRIZES PARALELA EM ERLANG

Theodoro Salazar Timmen
Eduardo Grohs Gottert

NOVO HAMBURGO
2025

1 INTRODUÇÃO

Este trabalho analisa o ganho de desempenho obtido ao paralelizar a soma de duas matrizes NxN em Erlang. A operação é naturalmente paralela, pois cada linha do resultado pode ser calculada de forma independente. Erlang foi escolhido por ter suporte nativo a processos leves que se comunicam por troca de mensagens, sem compartilhar memória — modelo conhecido como modelo de atores.

2 PROPOSTA DO EXPERIMENTO

2.1 Problema e hipótese

O objetivo é comparar o tempo da soma sequencial com o da versão paralela, variando número de workers (1, 2, 4, 8 e 16) e tamanho das matrizes (100×100 até 1000×1000). A hipótese é que, para matrizes grandes, mais workers diminuem o tempo até um ponto ótimo — após esse ponto, o custo de criar processos e copiar dados passa a superar o ganho. Para matrizes pequenas, o paralelo deve ser mais lento desde o início.

2.2 Variáveis

Independentes: número de workers e tamanho N. **Dependentes:** tempo de execução (ms), $\text{Speedup} = T_{\text{seq}}/T_{\text{par}}$ e $\text{Eficiência} = (\text{Speedup}/\text{Workers}) \times 100\%$. **Controle:** hardware e Erlang/OTP 13.2.2.5, fixos em todas as execuções.

3 IMPLEMENTAÇÃO

O módulo `matrix_sum.erl` tem duas versões: a sequencial usa `lists:zipwith/3` para somar os elementos linha por linha (versão de referência); a paralela divide as linhas em *W* partes com `lists:split/2`, cria *W* processos com `spawn/1` e cada um soma sua fatia devolvendo o resultado por mensagem (!). O processo principal coleta tudo com `receive`, ordena e concatena. O tempo é medido com `erlang:monotonic_time(microsecond)`, com média de três execuções por configuração.

4 EXECUÇÃO E COLETA DE DADOS

Ambiente: Ubuntu 24.04, Erlang/OTP 13.2.2.5 (BEAM SMP). A tabela abaixo apresenta resultados selecionados para os tamanhos mais representativos (N=100, 500 e 1000), com configurações de 1, 4, 8 e 16 workers.

Tabela 1 – Tempos de execução, Speedup e Eficiência por configuração

N	Workers	T. Seq. (ms)	T. Par. (ms)	Speedup	Efic. (%)
100×100	1	0.277	0.859	0.323	32.3
	4		0.900	0.308	7.7
	16		0.691	0.401	2.5
500×500	1	58.480	85.210	0.686	68.6
	4		53.580	1.091	27.3
	8		32.600	1.794	22.4
	16		40.890	1.430	8.9
1000×1000	1	481.200	388.600	1.238	123.8
	2		233.600	2.060	103.0
	4		199.400	2.414	60.3
	8		228.200	2.109	26.4
	16		199.800	2.408	15.1

Fonte: elaborada pelos autores a partir dos dados coletados (2025).

Valores de eficiência acima de 100% (N=1000, workers 1 e 2) ocorrem porque o processo worker opera em heap isolado, podendo reduzir a pressão sobre o coletor de lixo do processo principal.

5 ANÁLISE DAS QUESTÕES

5.1 Impacto da paralelização

Para N=100 (~0,28 ms sequencial) o paralelo foi sempre mais lento: o custo de criar processos superou qualquer ganho. Para N=1000 (~481 ms sequencial), a paralelização reduziu o tempo em até 58%: com 4 workers o tempo caiu para ~199 ms (speedup 2,41×).

5.2 Linearidade do ganho

Não foi linear. Em N=1000: de 1 para 2 workers o speedup subiu de 1,24 para 2,06; de 2 para 4 subiu apenas para 2,41; de 4 para 8 regrediu para 2,11. As etapas sempre sequenciais — dividir linhas, criar processos e juntar resultados — limitam o ganho total.

5.3 Influência do modelo Erlang

Processos Erlang têm memória separada: ao devolver o resultado, os dados são copiados integralmente. Em matrizes grandes com muitos workers, isso aumenta o tempo de comunicação. A vantagem é que elimina travas e condições de corrida por construção, sem nenhuma sincronização manual.

5.4 Principais gargalos

- Criação dos processos: custo fixo por worker, dominante para matrizes pequenas.
- Cópia dos dados nas mensagens: cresce com o tamanho da matriz e com o número de workers.
- Junção sequencial dos resultados ao final.
- Disputa por CPU com 16 workers em máquina com poucos núcleos físicos.

5.5 Limite prático do paralelismo

Para $N=1000$, o melhor resultado foi com 4 workers (speedup 2,41). Com 8 e 16 workers o desempenho estabilizou ou piorou. Para $N \leq 300$, qualquer quantidade de workers foi mais lenta que o sequencial. O paralelismo só valeu a pena a partir de $N=500$.

5.6 Possíveis melhorias

- Usar ETS ou NIFs para reduzir a cópia de dados nas mensagens.
- Cada worker retornar apenas os valores somados, sem a linha inteira.
- Executar em hardware físico multi-core dedicado.
- Testar matrizes maiores ($N=2000$, 5000) e usar eprof para profiling detalhado.

6 CONCLUSÃO

O experimento confirmou a hipótese. Para matrizes grandes, a soma paralela em Erlang reduziu o tempo em até 58% com 4 workers. Para matrizes pequenas, o overhead de criação de processos e cópia de mensagens superou o ganho. O principal limitador foi a cópia de dados entre heaps isolados, característica do modelo de atores. O paralelismo em Erlang é eficaz quando o trabalho por processo é grande o suficiente para compensar os custos de comunicação.

REFERÊNCIAS

- ERLANG/OTP. `erlang:monotonic_time/1`. Disponível em: https://www.erlang.org/doc/man/erlang.html#monotonic_time-1. Acesso em: 8 jun. 2025.
- ERLANG/OTP. `erlang:spawn/1`. Disponível em: <https://www.erlang.org/doc/man/erlang.html#spawn-1>. Acesso em: 8 jun. 2025.
- ERLANG/OTP. Erlang Efficiency Guide: processos. Disponível em: https://www.erlang.org/doc/efficiency_guide/processes.html. Acesso em: 8 jun. 2025.
- ERLANG/OTP. Erlang Reference Manual: processos. Disponível em: https://www.erlang.org/doc/reference_manual/processes.html. Acesso em: 8 jun. 2025.
- ERLANG/OTP. `lists:split/2`. Disponível em: <https://www.erlang.org/doc/man/lists.html#split-2>. Acesso em: 8 jun. 2025.

ERLANG/OTP. lists:zipwith/3. Disponível
<https://www.erlang.org/doc/man/lists.html#zipwith-3>. Acesso em: 8 jun. 2025.

em: